From Credit Density to Creative Entropy: Harnessing Intermediate Errors in Latent-Reasoning Language Models

Kevin T. N.
jkdkr2439@gmail.com

February 2026

Abstract

Latent-reasoning language models (LRLMs) perform multiple internal refinement steps before emitting each token. Recent work on Reward Latent Thought Trajectories (RLTT) shows that reinforcement learning objectives for looped language models should credit the *entire* latent refinement trajectory rather than only the terminal latent state, yielding denser credit assignment and improved reasoning accuracy and token efficiency. This paper develops a complementary thesis: *credit densification is a compression of exploration that can narrow the model's space of plausible internal solution drafts, thereby reducing controlled novelty if applied indiscriminately.* We formalize this trade-off using two operational constructs: (i) *credit density*, the effective learning signal delivered to early internal decisions; and (ii) *creative entropy*, the *constrained diversity* of internal trajectories that remain verifiably correct (or repairable under a bounded budget). We then propose ErrorTrace, a protocol that logs intermediate failures and reuses them as structured exploration seeds under explicit correctness constraints. The result is a design recipe for systems that can be reliably correct under strong ground truth while still supporting controlled exploratory behavior for hypothesis generation and creative search.

Keywords: latent reasoning; reinforcement learning; credit assignment; reward shaping; exploration; diversity; verification; hallucination; looped language models.

1 Introduction

Latent-reasoning architectures allow a model to allocate additional computation per output token without increasing output length. Looped Language Models (LoopLMs) instantiate this by repeatedly applying shared-weight transformer blocks to refine internal representations before emitting each token. When reinforcement learning is applied with verifiable rewards (RLVR), a natural question arises: *should reward credit the internal refinement process or only the final emission step?*

RLTT answers this for LoopLMs: terminal-only objectives such as GRPO implicitly treat each output token as a single-step decision and therefore bottleneck credit assignment, whereas trajectory-level credit aligns reinforcement learning with multi-step latent computation by directly rewarding the next-token distribution produced at every loop [1].

This paper agrees with RLTT's core insight and asks a second question: *What is lost when reward is made dense across internal reasoning?* In many settings (creative ideation, scientific hypothesis generation, open-ended design), intermediate "wrong" drafts can encode useful decompositions, surprising analogies, and near-miss structures. If dense credit pressures internal trajectories to stabilize too early, it can reduce the diversity of plausible internal drafts even when final correctness improves. The tension is thus not "correctness vs. randomness," but *correctness vs. constrained exploration capacity.*

Remark 1 (Metaphor only)**.** *A helpful intuition is the "frequent measurement" metaphor: rewarding every internal step can act like a strong constraint applied early and repeatedly, causing internal drafts to converge quickly. This is* not *a physical claim about quantum mechanics; it is a structural analogy about constraint pressure.*

Scope and stance

The paper is conceptual and operational: it introduces measurable constructs and an evaluation protocol. Claims about "creativity" are explicitly *protocol-relative* (ground truth defined by verifiers and rubrics), not metaphysical claims about intrinsic creativity.

2 Background: RLTT and Trajectory-Level Credit

LoopLMs compute, for each output token position j, a latent refinement trajectory $h_j^{(1)} \to h_j^{(2)} \to \cdots \to h_j^{(T_{\max})}$, and a corresponding trajectory of next-token distributions $P_\theta^{(1)}(\cdot \mid x, y_{<j}) \to \cdots \to P_\theta^{(T_{\max})}(\cdot \mid x, y_{<j})$. RLTT replaces terminal-only policy gradients (which depend directly on $P_\theta^{(T_{\max})}$ only) with a weighted sum over loops [1]. Intuitively: reward should shape the full latent thought distribution trajectory, not only its endpoint.

3 Reference Conditions and Technical Ground Truth

Many disputes about "better training" conflate changes in task, verifier, or budget. We therefore index claims to explicit reference conditions.

Definition 1 (Reference condition)**.** *A* reference condition *is a tuple*

$$\mathcal{C} = (\mathcal{T}, B, \mathcal{R}, \mathcal{V}, \mathcal{D}),$$

where $\mathcal{T}$ is a task distribution, B is a compute budget (tokens, loops, wall-clock), $\mathcal{R}$ is a reward specification, $\mathcal{V}$ is a verifier/evaluation protocol, and $\mathcal{D}$ is the data regime.

3.1 Ground truth tiers (operational)

We distinguish three practical tiers of ground truth, ordered by auditability:

1. **Programmatic ground truth:** exact verifiers (unit tests, compilers, theorem provers, numerical checks).
2. **Protocol ground truth:** controlled human evaluation with rubric, multiple raters, and adjudication. Reliability should be reported (e.g., Krippendorff's α or Cohen's κ) or an equivalent agreement procedure.
3. **Proxy ground truth:** automated judges or heuristics with known failure modes; useful for screening but insufficient for final claims.

This paper's key metrics are designed for tiers (1) and (2).

4 Two Constructs: Credit Density and Creative Entropy

4.1 Credit density (ID/DEF-bound to loop-wise objectives)

We now define credit density so it is directly observable in LoopLM/RLTT-style training where per-loop log-probabilities are recorded.

Let the token-level trajectory-credit objective decompose into per-loop contributions:

$$J(\theta) \;=\; \mathbb{E}\Big[\sum_{t=1}^{T_{\max}} J_t(\theta)\Big] \;-\; \beta\,\mathbb{E}\big[\mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})\big],$$

where $J_t(\theta)$ is the loop-t term (e.g., weighted advantage times per-loop log-probability; cf. RLTT [1]).

Definition 2 (Loop-t gradient contribution)**.** *Define the loop-t gradient contribution as*

$$g_t(\theta) \;=\; \nabla_\theta\,\mathbb{E}[J_t(\theta)].$$

In implementation, g_t can be approximated by the stochastic gradient of the loop-t loss term computed from recorded per-loop log-probabilities.

Definition 3 (Credit density)**.** *Fix a reference condition $\mathcal{C}$ and a LoopLM with $T_{\max}$ loops. Define* credit density *as*

$$D(\mathcal{C}) \;=\; \frac{1}{T_{\max}} \sum_{t=1}^{T_{\max}} w(t)\,\mathbb{E}[\|g_t(\theta)\|]\,,$$

where $w(t) \geq 0$ emphasizes early loops when desired (e.g., $w(t) \propto 1/t$) and the expectation is over the training distribution and policy rollouts under $\mathcal{C}$.

Remark 2. *Terminal-only credit tends to reduce $\|g_t\|$ for early loops because reward must backpropagate indirectly through the terminal distribution. Trajectory-level credit increases $D(\mathcal{C})$ by making early-loop gradients directly reward-shaped.*

4.2 Creative entropy (constrained diversity with verifiable plausibility)

"Entropy" alone can mean noise. We therefore define creativity as *constrained diversity*: diversity among trajectories that are verifiably correct or repairable under budget.

Let $\mathcal{H}$ denote a space of *internal trajectories* accessible for sampling (e.g., loop-wise draft distributions, intermediate decoded drafts, or any model-exposed internal "draft" representation).

Let $\mathcal{V}$ be a verifier (programmatic) or protocol (human rubric). Let $\textsc{Repair}_B$ be a bounded repair procedure consuming at most budget B (e.g., limited self-correction, limited search, or tool-assisted patching).

Definition 4 (Repairable plausibility indicator)**.** *For a sampled trajectory $h \in \mathcal{H}$, define its decoded output $y = \textsc{Decode}(h)$ and*

$$\pi_B(h) \;=\; \mathbf{1}\{\mathcal{V}(y) = 1\} \;\vee\; \mathbf{1}\{\mathcal{V}(\textsc{Repair}_B(y)) = 1\}\,.$$

Thus $\pi_B(h) = 1$ iff the trajectory is either immediately verifiable or repairable under budget B.

Definition 5 (Creative entropy (operational))**.** *Fix* $(x, \mathcal{C})$ *and sample* N *trajectories* $h_1, \ldots, h_N \sim \pi_\theta(\cdot \mid x)$ *under the same decoding/loop policy. Let* $\mathcal{S}(x) = \{h_i : \pi_B(h_i) = 1\}$ *be the* plausible set*. Cluster plausible trajectories into* K *distinct trajectory-types (e.g., by edit distance over drafts or embedding distance), yielding empirical cluster frequencies* $\hat{p}_1, \ldots, \hat{p}_K$ *on* $\mathcal{S}(x)$*. Define*

$$\widehat{E}_{cre}(x; \mathcal{C}) \;=\; -\sum_{k=1}^{K} \hat{p}_k \log \hat{p}_k.$$

Interpretation. $\widehat{E}_{\text{cre}}$ measures how many *distinct, plausibly successful* internal approaches the model retains under fixed reference conditions. It is not "be random"; it is "keep multiple repairable/correct modes alive."

5 Core Hypotheses (Trade-Off Frontier)

Hypothesis 1 (Efficiency gain under strong verifiers)**.** *For tasks with programmatic ground truth, increasing credit density* $D(\mathcal{C})$ *improves convergence speed and token/loop efficiency of verified success, holding other factors fixed.*

Hypothesis 2 (Premature convergence risk)**.** *For exploration-heavy tasks, increasing* $D(\mathcal{C})$ *without explicit exploration preservation reduces* $\widehat{E}_{cre}(x; \mathcal{C})$*, increasing the risk of premature trajectory collapse.*

Hypothesis 3 (Error reuse restores constrained diversity)**.** *Logging and reusing intermediate failures as structured exploration seeds can increase* $\widehat{E}_{cre}(x; \mathcal{C})$ *while maintaining verified accuracy, under fixed budget.*

6 Method: ErrorTrace (Reuse Intermediate Errors Safely)

The method treats intermediate failures as hypotheses rather than waste.

6.1 Error Trace Buffer (ETB)

For each episode/prompt, record:

$$\text{ETB} \ni \Big(x,\ \{h^{(t)}\}_{t=1}^{T_{\max}},\ y,\ r,\ \mathcal{V}(y),\ \text{diag}\Big),$$

where diag includes failure type, repair distance, compute usage, and loop at which divergence began.

6.2 Training objective (correctness + gated novelty)

Let $J_{\text{traj}}(\theta)$ be a trajectory-credit objective (e.g., RLTT-like). Define a novelty measure $\Delta_{\text{nov}}(h)$ computed *among plausible trajectories only* (e.g., distance to nearest plausible neighbor). Train with:

$$J(\theta) \;=\; J_{\text{traj}}(\theta) \;+\; \lambda\, \mathbb{E}\big[\pi_B(h)\, \Delta_{\text{nov}}(h)\big] \;-\; \beta\, \mathbb{E}\big[\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})\big].$$

The gate $\pi_B(h)$ prevents "novelty by nonsense."

Algorithm 1 ErrorTrace inference (high level)

[1]

Prompt x, budget B, verifier $\mathcal{V}$, candidates K, model π_θ, buffer ETB

$\mathcal{C} \leftarrow \emptyset$

for $k = 1$ to K **do**

  $s_k \leftarrow \textsc{SeedFromETB}(x, \text{ETB})$ ▷ near-miss patterns, failure motifs

  $h_k \sim \pi_\theta(\cdot \mid x, s_k)$

  $y_k \leftarrow \textsc{Decode}(h_k)$

  $\mathcal{C} \leftarrow \mathcal{C} \cup \{(h_k, y_k)\}$

end for

for all $(h_k, y_k) \in \mathcal{C}$ **do**

  if $\mathcal{V}(y_k) = 1$ **then**

    return y_k

  else

    $y'_k \leftarrow \textsc{Repair}_B(y_k)$

    if $\mathcal{V}(y'_k) = 1$ **then**

      return y'_k

    end if

  end if

end for

return $\textsc{BestEffort}(\mathcal{C})$

6.3 Inference: explore with error seeds, certify with verifiers

7 Evaluation Plan (Ground-Truth-First)

7.1 Task families

Use both high-auditability and protocol-evaluated tasks:

- **Math (exact):** verified by exact match; optional symbolic checking.
- **Code (tests):** verified by unit tests and compilation.
- **Formal reasoning:** verified by proof checker (when available).
- **Ideation/design:** protocol ground truth with rubric + multiple raters + reliability report.

7.2 Metrics

Reliability: verified accuracy; repair success under fixed B; verified success per unit compute.
Constrained exploration: $\widehat{E}_{\text{cre}}$; number of distinct plausible clusters; productive error rate (fraction of ETB failures that become verified successes after bounded repair).

7.3 Ablations

1. Terminal-only credit vs. trajectory credit (RLTT-style), fixed decoding and KL.
2. Trajectory credit with/without gated novelty.
3. ETB enabled vs. disabled (and retrieval on/off).
4. Fixed correctness, vary λ to map a Pareto frontier (accuracy vs. constrained diversity).

8 Discussion

8.1 Why this is not "just increase temperature"

Temperature increases surface-level entropy but can inflate error without preserving plausible modes. ErrorTrace preserves exploration by keeping diversity inside the verifiable/repairable set.

8.2 Safety note

When verifiers are weak (proxy ground truth), novelty incentives can amplify confident nonsense ("reward hacking" dynamics). Exploration bonuses should be used primarily under strong verifiers or robust human protocols with reliability.

8.3 Limitations

Internal trajectory access may require architectural support (loop-wise distributions or draft extraction). Protocol ground truth is expensive. Diversity metrics can be gamed and must be audited.

9 Conclusion

RLTT-style trajectory credit improves learning in LoopLMs by densifying credit assignment. But densification is also a form of constraint pressure that can reduce the diversity of plausible internal drafts. We introduced two operational constructs—credit density and constrained creative entropy—and proposed ErrorTrace, a practical protocol that logs intermediate errors and reuses them as structured exploration seeds under explicit correctness constraints. The core design rule is:

> *Use dense credit to secure correctness, but preserve controlled exploration by treating intermediate failures as a hypothesis library that can be verified or repaired under budget.*

Acknowledgements

None.

References

[1] J. Williams and E. Tureci. *Prioritize the Process, Not Just the Outcome: Rewarding Latent Thought Trajectories Improves Reasoning in Looped Language Models.* arXiv:2602.10520v2, 2026. https://arxiv.org/abs/2602.10520.

[2] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[4] Z. Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

[5] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.

[6] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.